JODHPUR INSTITUTE OF ENGINEERING AND TECHNOLOGY

Jodhpur, Rajasthan

PROJECT REPORT

On

Parallel Image Processing System

Submitted in partial fulfillment for the subject
Distributed Algorithm Lab (Hadoop) [6CS4-24]

Submitted By:

Neeraj Kumhar
Roll No: 24EJICS201
Registration No: JIET24CSD003
Semester: 6th (3rd Year)
Branch: Computer Science Engineering
Section: A3

Submitted To:

Dr. Naresh Purohit
HOD, CSE Department

Department of Computer Science Engineering
JIET - Jodhpur Institute of Engineering & Technology
Academic Session 2025-26

Acknowledgement

I would like to express my sincere gratitude to my subject teacher **Dr. Naresh Purohit, Head of the Department (CSE)**, Jodhpur Institute of Engineering and Technology, for her valuable guidance and continuous support during the development of this project.

I would also like to thank **Mrs. Anju Jangid, Head of the Department (CSE)** for providing the necessary facilities and academic environment required to complete this project successfully.

I am also grateful to my classmates and friends for their cooperation and helpful discussions during the implementation and testing of this project.

Finally, I would like to thank my family members for their constant encouragement and support throughout my academic journey.

Neeraj Kumhar
Roll No: 24EJICS201
Registration No: JIET24CSD003

Abstract

With the rapid growth of digital media, image processing has become an essential component in many applications such as medical imaging, computer vision, surveillance systems, and multimedia systems. Processing high-resolution images often requires significant computational power and time. Traditional sequential processing techniques may become inefficient when dealing with large image datasets.

The project titled **Parallel Image Processing System** aims to demonstrate the use of parallel computing techniques to improve the performance of image processing operations. The system allows users to upload images through a web-based interface and apply various image filters such as grayscale conversion, blur filtering, and edge detection. The image processing tasks are executed using parallel computing techniques implemented through the **OpenMP (Open Multi-Processing)** library in C++.

OpenMP is a widely used API that supports multi-threaded parallel programming in shared memory architectures. By using OpenMP directives such as `#pragma omp parallel for`, the system divides image processing tasks across multiple CPU cores, allowing multiple parts of the image to be processed simultaneously.

The frontend of the system is developed using **React.js**, which provides an interactive and user-friendly interface for uploading images and viewing processing results. The backend is implemented using **C++ with OpenMP and OpenCV**. The backend receives images through API requests, processes them in parallel using multiple threads, and returns the processed images to the frontend.

The system also measures and displays the processing time, allowing users to observe the performance improvements achieved through parallel computing compared to sequential processing.

This project demonstrates the practical application of distributed and parallel computing concepts in image processing systems. It provides an educational example of how parallel algorithms can significantly improve computational efficiency when working with large-scale data.

Contents

1 Introduction **5**
1.1 Objectives 5

2 Technologies Used **7**
2.1 Frontend Technologies 7
2.1.1 React.js 7
2.1.2 Axios 7
2.1.3 TailwindCSS 7
2.1.4 JavaScript, HTML, and CSS 8
2.2 Backend Technologies 8
2.2.1 C++ Programming Language 8
2.2.2 OpenMP (Open Multi-Processing) 8
2.2.3 OpenCV (Open Source Computer Vision Library) 8
2.2.4 HTTP Server (cpp-httplib / Crow) 9
2.3 Summary 9

3 System Architecture **10**
3.1 System Architecture Diagram 10
3.2 User Interface of the System 11
3.3 Workflow of the System 11

4 Core Implementation Code **13**
4.1 Grayscale Conversion using OpenMP 13
4.2 Blur Filter Implementation 14
4.3 Edge Detection using Sobel Operator 15
4.4 Backend API / Server Implementation 16

5 Implementation **17**
5.1 Grayscale Conversion 17
5.2 Blur Filter 18
5.3 Edge Detection 18
5.4 Parallel Processing Strategy 19
5.5 Performance Improvement 19

6 Advantages **20**

7 Limitations **21**

8 Future Scope **22**

9 Conclusion **23**

Chapter 1

Introduction

Image processing is a field of computer science that focuses on analyzing and manipulating digital images to extract useful information or enhance visual quality. Image processing techniques are widely used in areas such as medical diagnostics, satellite imaging, facial recognition systems, autonomous vehicles, and multimedia applications.

However, processing high-resolution images often involves a large number of computations because each pixel of the image must be processed individually. For example, a Full HD image with a resolution of 1920×1080 contains more than two million pixels. Applying filters or transformations to each pixel sequentially can be time-consuming and computationally expensive.

Parallel computing provides an effective solution to this problem by dividing a large task into smaller sub-tasks that can be processed simultaneously using multiple CPU cores. By executing operations concurrently, parallel computing significantly reduces execution time and improves overall system performance.

The **Parallel Image Processing System** developed in this project demonstrates how parallel programming techniques can be applied to image processing operations. The system allows users to upload images through a web interface and apply different filters. The backend processes the image using **OpenMP parallel programming in C++**, which distributes the workload across multiple processor cores.

The frontend interface is implemented using **React.js**, which provides a responsive and interactive platform for uploading images and displaying the processed results. The backend server performs the computational tasks using **OpenCV** for image manipulation and **OpenMP** for parallel execution.

This project provides a practical demonstration of parallel algorithms and highlights the advantages of multi-threaded processing in modern computing systems.

1.1 Objectives

The main objectives of the Parallel Image Processing System are:

- To understand the concept of parallel computing using OpenMP.
- To implement image processing algorithms using parallel programming techniques.
- To develop a web-based system that allows users to upload and process images.
- To reduce image processing time using multi-threaded execution.

- To demonstrate the performance difference between sequential and parallel processing.
- To gain practical experience in distributed and parallel algorithm implementation.

Chapter 2

Technologies Used

The development of the Parallel Image Processing System involves a combination of modern frontend and backend technologies. These technologies are carefully selected to ensure efficient user interaction, high-performance computation, and seamless communication between system components. The frontend is responsible for providing an intuitive interface, while the backend handles computationally intensive image processing tasks using parallel programming techniques.

2.1 Frontend Technologies

The frontend of the system is designed to provide an interactive and user-friendly interface through which users can upload images, select processing filters, and visualize results.

2.1.1 React.js

React.js is a widely used JavaScript library for building dynamic and responsive user interfaces. It follows a component-based architecture, which allows developers to create reusable UI components and efficiently manage application state.

In this project, React is used to design the main user interface, including image upload panels, filter selection controls, and result display sections. React's virtual DOM mechanism ensures fast rendering and improved performance, making it suitable for real-time interaction in web applications.

2.1.2 Axios

Axios is a promise-based HTTP client used for making asynchronous requests between the frontend and backend. It simplifies API communication by handling request and response operations efficiently.

In this system, Axios is used to send image data and selected filter parameters from the React frontend to the C++ backend server. It also receives the processed image and performance metrics such as processing time and thread count.

2.1.3 TailwindCSS

TailwindCSS is a utility-first CSS framework that enables rapid UI development using predefined styling classes. It eliminates the need for writing extensive custom CSS by

providing ready-to-use utility classes for layout, spacing, typography, and responsiveness.

In this project, TailwindCSS is used to create a clean, modern, and responsive interface. It enhances the visual appearance of the application while ensuring consistency across different screen sizes and devices.

2.1.4 JavaScript, HTML, and CSS

JavaScript, HTML, and CSS form the foundation of the frontend system. HTML is used to structure the content, CSS is used for styling and layout, and JavaScript is used to implement dynamic behavior and user interactions.

These technologies work together with React to provide a seamless and interactive user experience.

2.2 Backend Technologies

The backend of the system is responsible for performing image processing operations and handling computational tasks efficiently using parallel programming techniques.

2.2.1 C++ Programming Language

C++ is a high-performance programming language widely used for system-level programming and computationally intensive applications. It provides low-level memory control and efficient execution, making it ideal for performance-critical tasks.

In this project, C++ is used to implement the core image processing logic. Its speed and efficiency make it suitable for handling large image data and performing pixel-level operations.

2.2.2 OpenMP (Open Multi-Processing)

OpenMP is an API that supports multi-threaded parallel programming in shared memory environments. It allows developers to parallelize code using compiler directives such as `#pragma omp parallel for`.

In this system, OpenMP is used to divide image processing tasks across multiple CPU cores. Each thread processes a portion of the image simultaneously, significantly reducing execution time compared to sequential processing. This demonstrates the practical application of parallel algorithms in improving system performance.

2.2.3 OpenCV (Open Source Computer Vision Library)

OpenCV is an open-source library widely used for image processing and computer vision applications. It provides a rich set of functions for reading, writing, and manipulating images.

In this project, OpenCV is used to load images, access pixel data, and apply various image processing operations such as grayscale conversion, blurring, and edge detection. Its optimized functions and ease of integration with C++ make it an essential component of the backend system.

2.2.4 HTTP Server (cpp-httplib / Crow)

A lightweight HTTP server library such as cpp-httplib or Crow is used to handle communication between the frontend and backend. It allows the backend to expose REST API endpoints for processing images.

The server receives image data from the frontend, processes it using OpenMP and OpenCV, and returns the processed output. This ensures smooth integration between the React frontend and the C++ backend.

2.3 Summary

The combination of React for frontend development and C++ with OpenMP for backend processing provides a powerful architecture for building high-performance web applications. While React ensures a responsive and user-friendly interface, C++ and OpenMP enable efficient parallel computation, making the system capable of handling large-scale image processing tasks effectively.

Chapter 3

System Architecture

The Parallel Image Processing System follows a client-server architecture where the frontend and backend work together to perform efficient image processing using parallel computing techniques.

The system is divided into two main components:

- **Frontend (Client Side):** Developed using React.js, it provides an interactive interface for users to upload images, select filters, and view results.
- **Backend (Server Side):** Implemented in C++ using OpenMP and OpenCV, it handles image processing tasks using parallel computation.

3.1 System Architecture Diagram

The following diagram illustrates the overall architecture of the Parallel Image Processing System. It shows how the frontend communicates with the backend server and how image processing is performed using parallel threads.

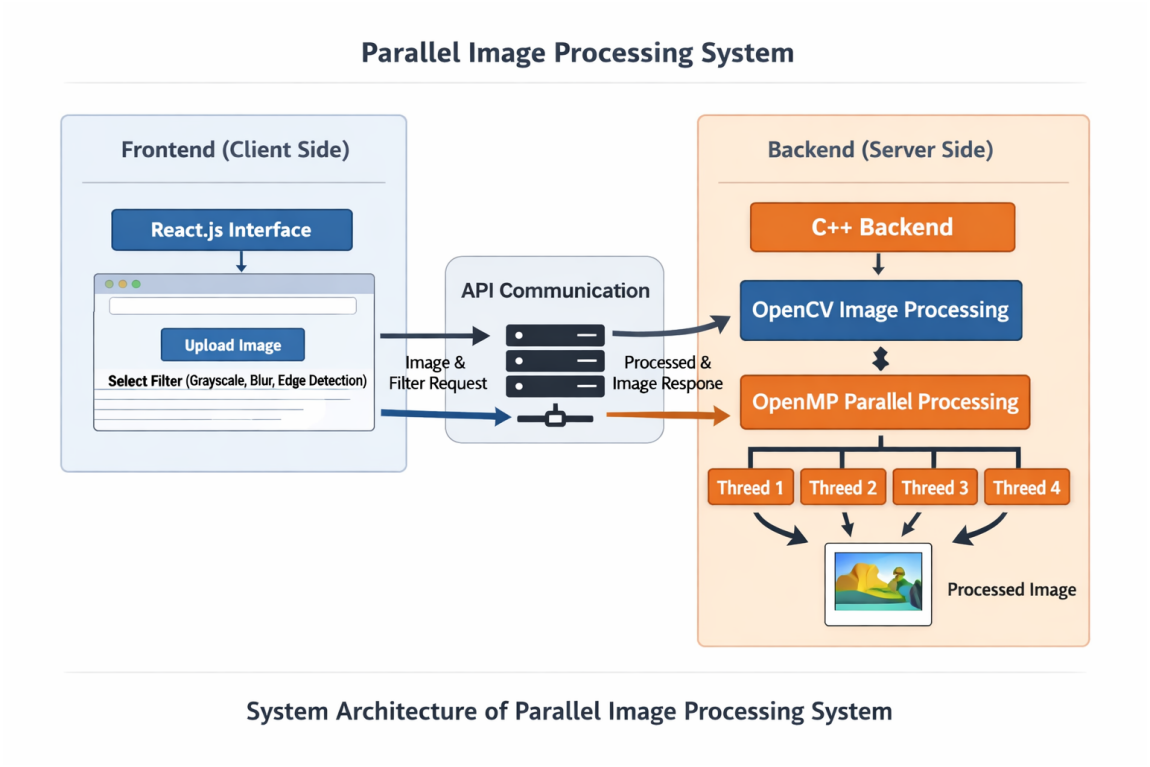

Figure 3.1: System Architecture of Parallel Image Processing System

3.2 User Interface of the System

The user interface of the system is designed to be simple, interactive, and user-friendly. It allows users to upload images, select image processing filters, and view the processed output side by side.

3.3 Workflow of the System

The working of the system can be explained through the following steps:

1. The user opens the web application and uploads an image through the React interface.
2. The user selects the desired image processing filter such as grayscale, blur, or edge detection.
3. The frontend sends the image and selected filter to the backend server using an API request.
4. The backend loads the image using OpenCV and processes it using OpenMP parallel threads.
5. Each thread processes a portion of the image simultaneously, improving performance.
6. The processed image is returned to the frontend.

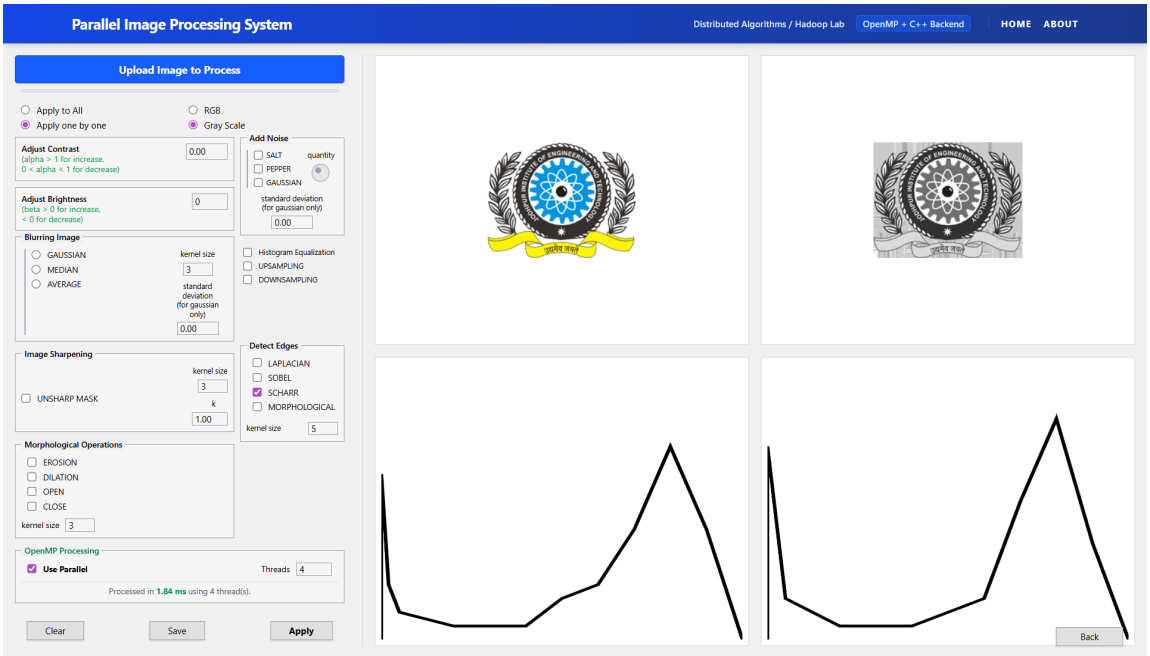

Figure 3.2: User Interface of Parallel Image Processing System

7. The frontend displays both the original and processed images for comparison.

Chapter 4

Core Implementation Code

This chapter presents the key implementation code used in the Parallel Image Processing System. The following figures show the important parts of the backend code, including image processing using OpenCV and parallel execution using OpenMP.

The code snippets demonstrate how image pixels are processed using multi-threading techniques to improve performance and reduce execution time.

4.1 Grayscale Conversion using OpenMP

The following figure shows the implementation of grayscale image conversion using OpenMP parallel loops. Each thread processes a portion of the image simultaneously.

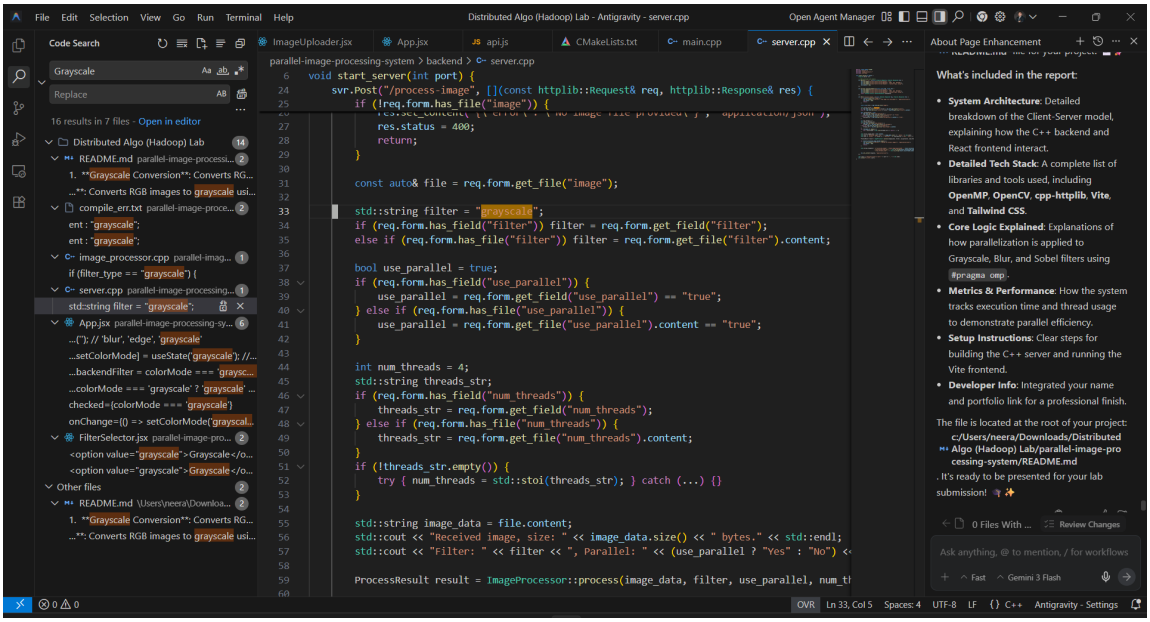

Figure 4.1: Grayscale Conversion using OpenMP

4.2 Blur Filter Implementation

The blur filter is implemented by averaging neighboring pixel values. The computation is parallelized using OpenMP to improve performance.

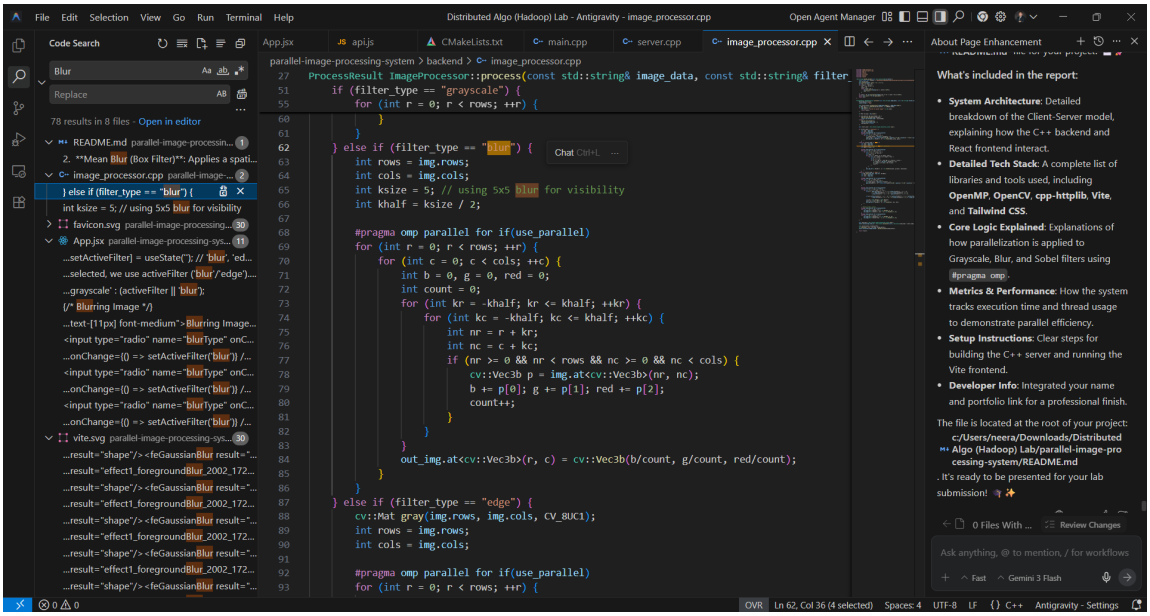

Figure 4.2: Blur Filter using Parallel Processing

4.3 Edge Detection using Sobel Operator

The following figure shows the implementation of edge detection using the Sobel operator. The gradients are computed in parallel using OpenMP.

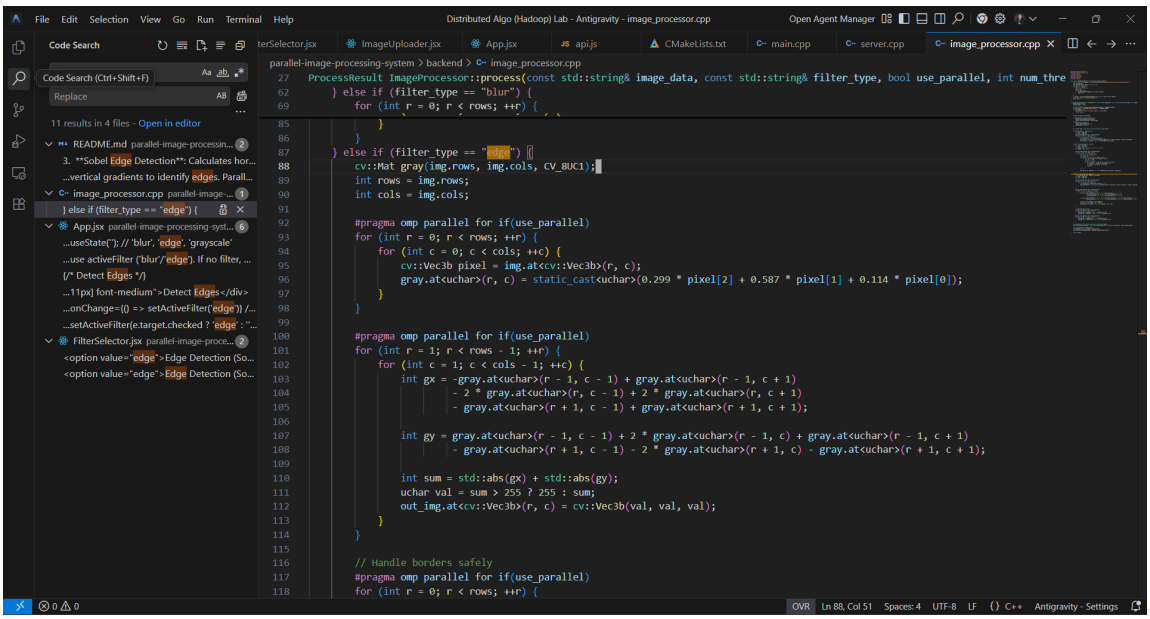

Figure 4.3: Edge Detection using Sobel Operator

4.4 Backend API / Server Implementation

This figure shows the backend server code responsible for handling image upload requests and returning processed images to the frontend.

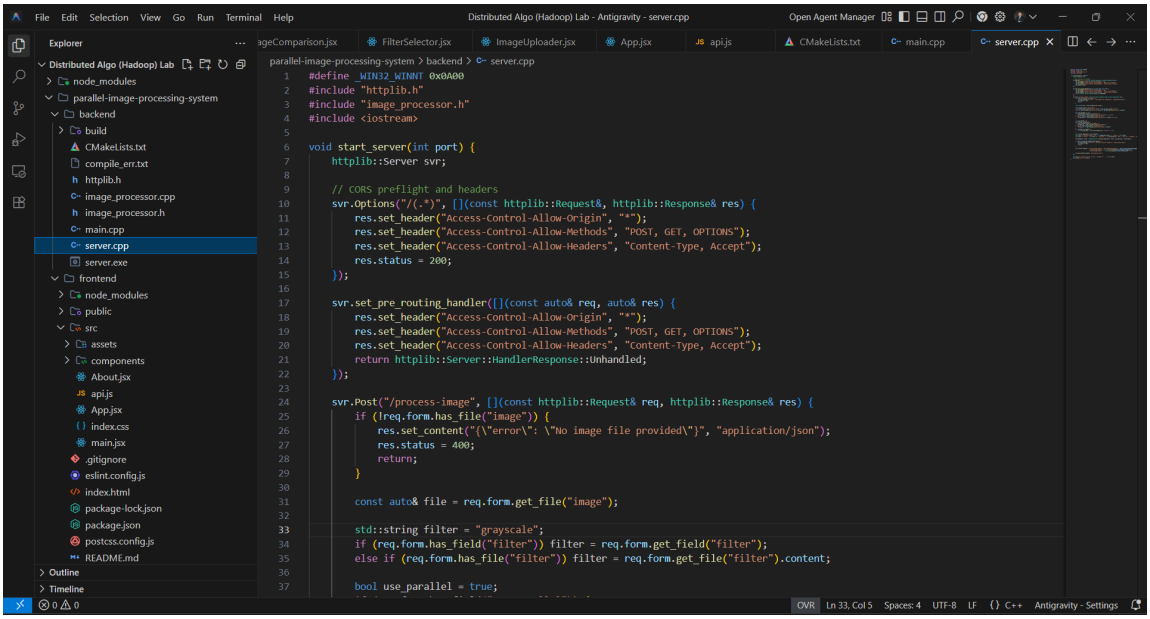

Figure 4.4: C++ Backend Server Implementation

Chapter 5

Implementation

The implementation of the Parallel Image Processing System focuses on applying various image processing techniques using parallel computing concepts. The system is designed to efficiently process images by dividing computational tasks into smaller sub-tasks and executing them simultaneously using multiple CPU threads through OpenMP.

The backend is implemented in C++ using OpenCV for image manipulation and OpenMP for parallel execution. The frontend, developed using React, allows users to interact with the system by uploading images and selecting desired filters. The backend processes the image and returns the output to the frontend, where it is displayed alongside the original image.

The implementation is divided into three major image processing operations: grayscale conversion, blur filtering, and edge detection.

5.1 Grayscale Conversion

Grayscale conversion is one of the fundamental image processing operations. It converts a colored image into a grayscale image by reducing the three color channels (Red, Green, Blue) into a single intensity value. This simplifies image analysis and reduces computational complexity.

The grayscale value of each pixel is calculated using the weighted sum of its RGB components as follows:

$$Gray = 0.299R + 0.587G + 0.114B$$

This formula is based on human perception, where the green channel contributes more to brightness perception compared to red and blue.

In the implementation, each pixel of the image is accessed using OpenCV's matrix representation. The conversion is performed by iterating over all pixels in the image.

To improve performance, OpenMP is used to parallelize the outer loop that iterates through image rows. Each thread processes a subset of rows independently, allowing multiple parts of the image to be converted simultaneously.

- The image is divided row-wise.
- Each thread processes a group of rows.
- The computation is performed concurrently across threads.

This significantly reduces the processing time compared to sequential execution, especially for high-resolution images.

5.2 Blur Filter

The blur filter is used to reduce noise and smooth the image by averaging the pixel values in a neighborhood. It replaces each pixel value with the average of its surrounding pixels, resulting in a softened image.

In this project, a simple mean blur filter is implemented. For each pixel, a small window (e.g., 3×3 kernel) is considered, and the average value of all pixels within that window is calculated.

The steps involved are:

1. Select a kernel window around each pixel.
2. Compute the average of neighboring pixel values.
3. Assign the computed value to the target pixel.

Parallelization is achieved by distributing the image rows among multiple threads using OpenMP. Each thread independently computes the blurred values for its assigned rows.

Key advantages of parallel implementation include:

- Faster processing of large images
- Efficient utilization of multi-core processors
- Reduced execution time compared to sequential methods

Care is taken to handle edge pixels properly, as they have fewer neighboring pixels.

5.3 Edge Detection

Edge detection is an important technique used to identify boundaries and significant transitions in an image. It is widely used in computer vision applications such as object detection, image segmentation, and pattern recognition.

In this project, the Sobel operator is used for edge detection. The Sobel operator calculates the gradient of the image intensity in both horizontal and vertical directions.

Two convolution kernels are used:

- Horizontal gradient (Gx)
- Vertical gradient (Gy)

The magnitude of the gradient is calculated as:

$$G = \sqrt{G_x^2 + G_y^2}$$

This value represents the strength of the edge at a particular pixel.

The implementation involves the following steps:

1. Apply Sobel kernels to compute horizontal and vertical gradients.
2. Calculate gradient magnitude for each pixel.
3. Normalize the result to fit within the image intensity range.

OpenMP is used to parallelize the computation by assigning different portions of the image to multiple threads. Each thread independently computes gradient values for its assigned pixels.

5.4 Parallel Processing Strategy

The core of this system lies in its parallel processing strategy. OpenMP directives such as:

```
#pragma omp parallel for
```

are used to distribute loop iterations across multiple threads.

The image is divided into segments (typically rows), and each segment is processed independently by a separate thread. This approach ensures:

- Efficient load balancing across threads
- Minimal thread synchronization overhead
- Improved computational performance

The number of threads can be controlled using OpenMP functions such as `omp_set_num_threads()`, allowing flexibility based on system capabilities.

5.5 Performance Improvement

The use of parallel processing significantly improves performance. For large images, the time required for processing is reduced as multiple threads work simultaneously.

The system also measures execution time for both sequential and parallel implementations, allowing users to observe the performance gain.

- Sequential Processing: Single-threaded execution
- Parallel Processing: Multi-threaded execution using OpenMP

Experimental results show that parallel processing reduces execution time and increases efficiency, demonstrating the practical benefits of parallel algorithms in real-world applications.

Chapter 6

Advantages

The Parallel Image Processing System offers several advantages in terms of performance, efficiency, usability, and practical implementation of parallel computing concepts. The key advantages of the system are described below:

- **Faster Image Processing:** One of the major advantages of the system is the significant reduction in processing time. By using OpenMP, the system divides the image processing task among multiple CPU threads, allowing different portions of the image to be processed simultaneously. This parallel execution leads to faster results compared to traditional sequential processing.
- **Efficient Utilization of Multi-Core Processors:** Modern computers are equipped with multi-core processors. This system effectively utilizes available CPU cores by distributing workload across threads, ensuring optimal use of hardware resources and improving overall system performance.
- **Scalable Architecture:** The system is designed to scale with increasing computational resources. As the number of CPU cores increases, the system can handle larger images and more complex processing tasks efficiently. This makes the system suitable for high-performance computing environments.
- **Interactive and User-Friendly Interface:** The React-based frontend provides a clean and intuitive interface that allows users to upload images, select filters, and view results easily. The side-by-side comparison of original and processed images enhances user understanding and interaction.
- **Demonstration of Parallel and Distributed Concepts:** The project effectively demonstrates key concepts of parallel computing and distributed algorithms. It provides practical exposure to thread-based execution, workload distribution, and performance optimization, which are essential topics in distributed systems.
- **Reduced Computational Bottlenecks:** By parallelizing pixel-level operations, the system minimizes bottlenecks associated with sequential execution. This is especially beneficial when processing high-resolution images with millions of pixels.
- **Flexibility and Extensibility:** The system can be easily extended to include additional image processing algorithms or integrated with other technologies such as machine learning and computer vision frameworks.

Chapter 7

Limitations

Although the Parallel Image Processing System provides significant advantages, there are certain limitations that may affect its performance and applicability in specific scenarios.

- **Dependency on Multi-Core Processors:** The performance improvement of the system depends heavily on the availability of multi-core processors. Systems with fewer cores may not experience significant speedup compared to parallel execution on high-end machines.
- **Memory Consumption:** Processing large images requires substantial memory resources. High-resolution images and parallel execution may increase memory usage, which can impact performance on systems with limited memory capacity.
- **Thread Management Overhead:** Although OpenMP simplifies parallel programming, the creation and management of threads introduce some overhead. For smaller images, the overhead may reduce the overall benefit of parallelization.
- **Limited to Shared Memory Systems:** OpenMP is designed for shared memory architectures, which limits the system to single-machine environments. It does not directly support distributed computing across multiple machines.
- **Network Latency in Web-Based Architecture:** Since the system operates as a web application, network delays may affect response time when transferring large image files between the frontend and backend.
- **Edge Handling Complexity:** Some image processing operations, such as blur and edge detection, require special handling for boundary pixels. Improper handling may lead to inaccurate results at image edges.

Chapter 8

Future Scope

The Parallel Image Processing System can be further enhanced by incorporating advanced features and technologies. Some potential future improvements are described below:

- **Advanced Image Processing Techniques:** Future versions of the system can include more sophisticated filters such as sharpening, histogram equalization, noise reduction, and morphological operations.
- **GPU-Based Parallel Processing:** The system can be extended to use GPU acceleration through technologies such as CUDA or OpenCL. GPUs offer massive parallelism and can significantly improve performance for large-scale image processing tasks.
- **Real-Time Video Processing:** The system can be upgraded to process video streams in real time, enabling applications in surveillance systems, object tracking, and live image analysis.
- **Cloud Integration:** The backend can be deployed on cloud platforms to handle large-scale image processing tasks. Integration with distributed computing frameworks can further enhance scalability and performance.
- **Machine Learning Integration:** The system can be integrated with machine learning models for advanced tasks such as object detection, facial recognition, and image classification.
- **User Authentication and Data Storage:** Future improvements may include user login systems, image history storage, and personalized dashboards for better user experience.
- **Hybrid Parallel Systems:** Combining OpenMP with distributed frameworks like MPI (Message Passing Interface) can enable both shared memory and distributed memory parallelism.

Chapter 9

Conclusion

The Parallel Image Processing System successfully demonstrates the practical implementation of parallel computing techniques in the field of image processing. The project highlights how computationally intensive tasks can be optimized using multi-threaded execution, resulting in improved performance and reduced processing time.

By utilizing OpenMP, the system efficiently distributes image processing operations across multiple CPU cores. This parallel execution significantly enhances the speed of operations such as grayscale conversion, blur filtering, and edge detection. The project also demonstrates how modern programming techniques can be applied to solve real-world problems involving large data processing.

The integration of a React-based frontend with a C++ backend provides a complete full-stack solution. The frontend offers an intuitive and user-friendly interface, while the backend performs high-performance computation using OpenCV and OpenMP. This combination ensures both usability and efficiency.

Furthermore, the project provides valuable insights into parallel programming concepts, including thread management, workload distribution, and performance optimization. It serves as an effective learning tool for understanding distributed and parallel algorithms in practical scenarios.

In conclusion, the Parallel Image Processing System achieves its objective of demonstrating the advantages of parallel computing in image processing applications. With further enhancements such as GPU acceleration and cloud integration, the system can be extended to handle more complex and large-scale computational tasks in real-world environments.